Chapter 1

Introduction to Machine Learning

AI ML

DM ⇒ Extracting knowledge from large databases.

Introduction to Machine Learning

What is Machine Learning?

Machine learning lies at the intersection of computer science, engineering, and statistics and often appears in other disciplines. As you'll see later, it can be applied to many fields from politics to geosciences. It's a tool that can be applied to many problems. Any field that needs to interpret and act on data can benefit from machine learning techniques. Machine learning uses statistics. To most people, statistics is an esoteric subject used for companies to lie about how great their products are

Key Tasks of ML/ Types of Machine Learning

Classification- To predict what class an instance of data should fall into.

Regression is the prediction of a numeric value. Most people have probably seen an example of regression with a best-fit line drawn through some data points to generalize the data points. Classification and regression are examples of supervised learning.

This set of problems is known as **supervised** because we're telling the algorithm what to predict. The opposite of supervised learning is a set of tasks known as **unsupervised learning.**

In unsupervised learning, there's no label or target value given for the data. A task where we group similar items together is known as **clustering.** In unsupervised learning, we may also want to find statistical values that describe the data. This is known as **density estimation.** Another task of unsupervised learning may be reducing the data from many features to a small number so that we can properly visualize it in two or three dimensions.

Supervised learning tasks	
k-Nearest Neighbors	Linear
Naive Bayes	Locally weighted linear
Support vector machines	Ridge
Decision trees	Lasso
Unsupervised learning tasks	
k-Means	Expectation maximization
DBSCAN	Parzen window

Table 1.2 **Common algorithms used to perform classification, regression, clustering, and density estimation tasks**

Scikit-learn · NumPy SciPy

How to choose the right algorithm?

First, you need to consider your goal. What are you trying to get out of this? (Do you want a probability that it might rain tomorrow, or do you want to find groups of voters with similar interests?) What data do you have or can you collect? Those are the big questions. Let's talk about your goal. If you're trying to predict or forecast a target value, then you need to look into supervised learning. If not, then unsupervised learning is the place you want to be. If you've chosen supervised learning, what's your target value? Is it a discrete value like Yes/No, 1/2/3, A/B/C, or Red/Yellow/Black? If so, then you want to look into classification. If the target value can take on a number of values, say any value from 0.00 to 100.00, or -999 to 999, or + to -, then you need to look into regression. If you're not trying to predict a target value, then you need to look into unsupervised learning. Are you trying to fit your data into some discrete groups? If so and that's all you need, you should look into clustering. Do you need to have some numerical estimate of how strong the fit is into each group? If you answer yes, then you probably should look into a density estimation algorithm.

You should spend some time getting to **know your data**, and the more you know about it, the better you'll be able to build a successful application. Things to know about your data are these: Are the features nominal or continuous? Are there missing values in the features? If there are missing values, why are there missing values? Are there outliers in the data? Are you looking for a needle in a haystack, something that happens very infrequently? All of these features about your data can help you narrow the algorithm selection process.

With the algorithm narrowed, there's no single answer to what the best algorithm is or what will give you the best results. **You're going to have to try different algorithms and see how they perform**. There are other machine learning techniques that you can use to improve the performance of a machine learning algorithm. The relative performance of two algorithms may change after you process the input data. We'll discuss these in more detail later, but the point is that **finding the best algorithm** is an iterative process of trial and error.

Steps in developing a machine learning application

1. **Collect data.** You could collect the samples by scraping a website and extracting data, or you could get information from an RSS feed or an API.
2. **Prepare the input data.** Once you have this data, you need to make sure it's in a useable format. You may need to do some algorithm-specific formatting here. Some algorithms need features in a special format, some algorithms can deal with target variables and features as strings, and some need them to be integers.
3. **Analyze the input data.** This is looking at the data from the previous task. This could be as simple as looking at the data you've parsed in a text editor to make sure steps 1 and 2 are actually working and you don't have a bunch of empty values.
4. **Train the algorithm.** This step and the next step are where the "core" algorithms lie, depending on the algorithm. You feed the algorithm good clean data from the first two steps and extract knowledge or information. This knowledge you often store in a format

that's readily useable by a machine for the next two steps. In the case of unsupervised learning, there's no training step because you don't have a target value. Everything is used in the next step.

5. **Test the algorithm.** This is where the information learned in the previous step is put to use. When you're evaluating an algorithm, you'll test it to see how well it does. In the case of supervised learning, you have some known values you can use to evaluate the algorithm. In unsupervised learning, you may have to use some other metrics to evaluate the success. In either case, if you're not satisfied, you can go back to step 4, change some things, and try testing again.
6. **Use it.** Here you make a real program to do some task, and once again you see if all the previous steps worked as you expected.

Applications of Machine Learning

1. **Face detection**: The face detection feature in mobile cameras is an example of what machine learning can do. Cameras can automatically snap a photo when someone smiles more accurately now than ever before because of advances in machine learning algorithms.
2. **Face recognition**: This is where a computer program can identify an individual from a photo. You can find this feature on Facebook for automatically tagging people in photos where they appear. Advances in machine learning means more accurate auto-face tagging softwares.
3. **Image classification**: A good example is the application of deep learning to improve image classification or image categorization in apps such as Google photos. Google photos would not be possible without advances in deep learning.
4. **Speech recognition**: Another good example is Google now. Improvements in speech recognition systems has been made possible by, you guessed right, machine learning specifically deep learning.
5. **Google**: Google defines itself as a machine learning company now. It is also a leader in this area because machine learning is a very important component to it's core advertising and search businesses. It applies machine learning to improve search results and search suggestions.
6. **Anti-virus:** Machine learning is used in Anti-virus softwares to improve detection of malicious software on computer devices.
7. **Anti-spam**: machine learning is also used to train better anti-spam software systems.
8. **Genetics:** Classical data mining or clustering algorithms in machine learning such as agglomerative clustering algorithms are used in genetics to help find genes associated with a particular disease.
9. **Signal denoising:** Machine learning algorithms such as the K-SVD which is just a generalization of k-means clustering are used to find a dictionary of vectors that can be sparsely linearly combined to approximate any given input signal. Thus such a technique is used in video compression and denoising.
10. **Weather forecast**: Machine learning is applied in weather forecasting software to improve the quality of the forecast.

Chapter 2

Learning with Regression

Refer video of London m/c learning study group

Ch. 2 - Regression.

- Applications.

1) Pattern Recognition.
   - Facial identities, medical images
   - Text recognition.

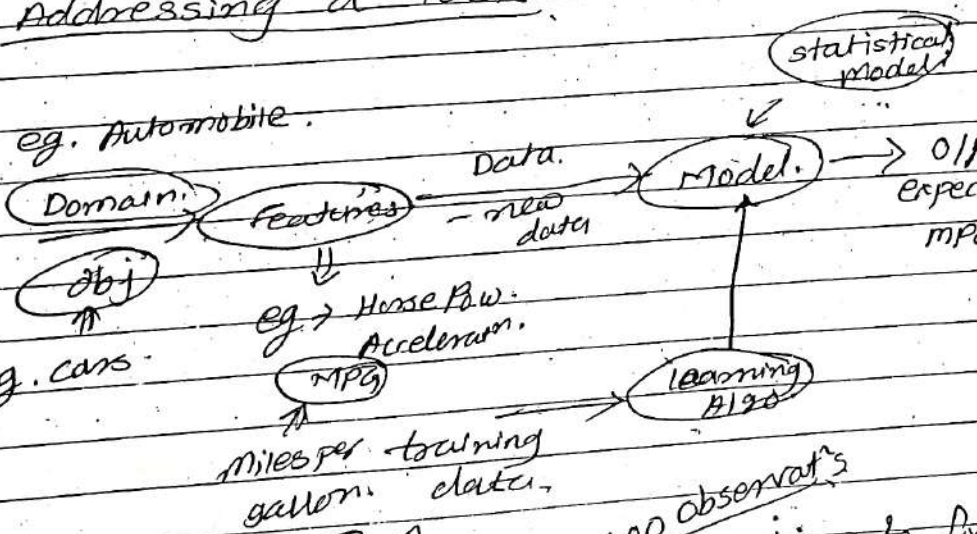
2) Prediction.
   - Stock Prices.
   - Marketing campaign.

3) Classificatn
   - Spam detectn.
   - Find similar content

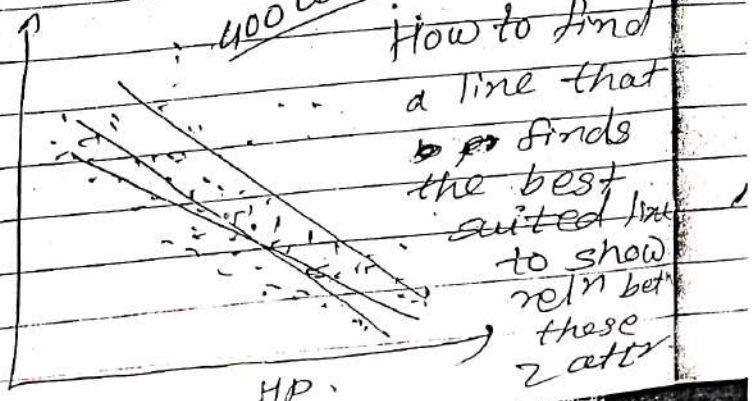
4) Anomaly Detectn.

- Addressing a task.

eg. Automobile.



Example



- Linear Regression (Supervised Algo).

- Models a single response (dependent, outcome) variable based on one or more i/p (independent, predictor) variables.

- Assume linear rel^n bet^n i/p & response variable.

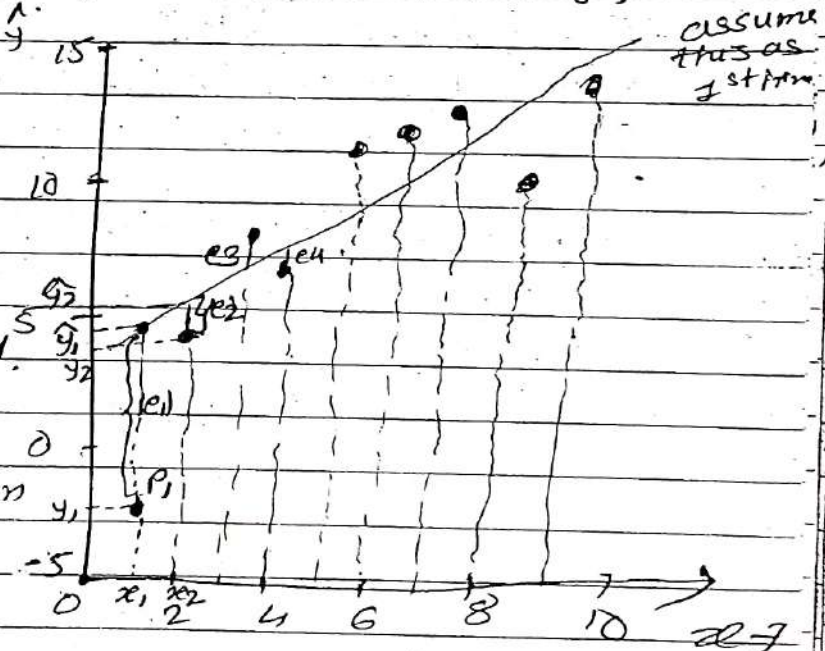
In our case, let, i/p is HP & response variable is MPG

Lets take a smaller dataset having only 9 observations.

$N = 9$.

$x = \{x_1, x_2 \cdots x_N\}$

$y = \{y_1, y_2 \cdots y_N\}$.



The aim is to find a func which enable us to make a predict^n for att^r y using att^r x.

~~Eq^n of line is $\alpha + \beta x$~~

$\hat{y}_i(x_i) = w_0 + w_1 x_i$

where it represents eq^n of a line $\alpha + \beta x$.

* How to find this line? or how to measure goodness of this line?

Eg → for point P_i in the graph, the actual distance is (x_i, y_i). But if we use line to give the distance, it gives us the y coordinate as $\hat{y}_i$, so the error betⁿ the actual distance & predicted distance is $|e_1|$. Similarly if we apply for all points, we get the error $|e_1|, |e_2| \ldots |e_n|$ for each of them.

So, the total error becomes,

$$E = |e_1| + |e_2| + \cdots + |e_N|$$

The best fitting line is the one which minimizes this error.

To do this, there is a cost func. in ml.

$$J(W) = \sum_{i=1}^{N} (\hat{y}_i - y_i)^2 \leftarrow \text{to ignore the sign.}$$

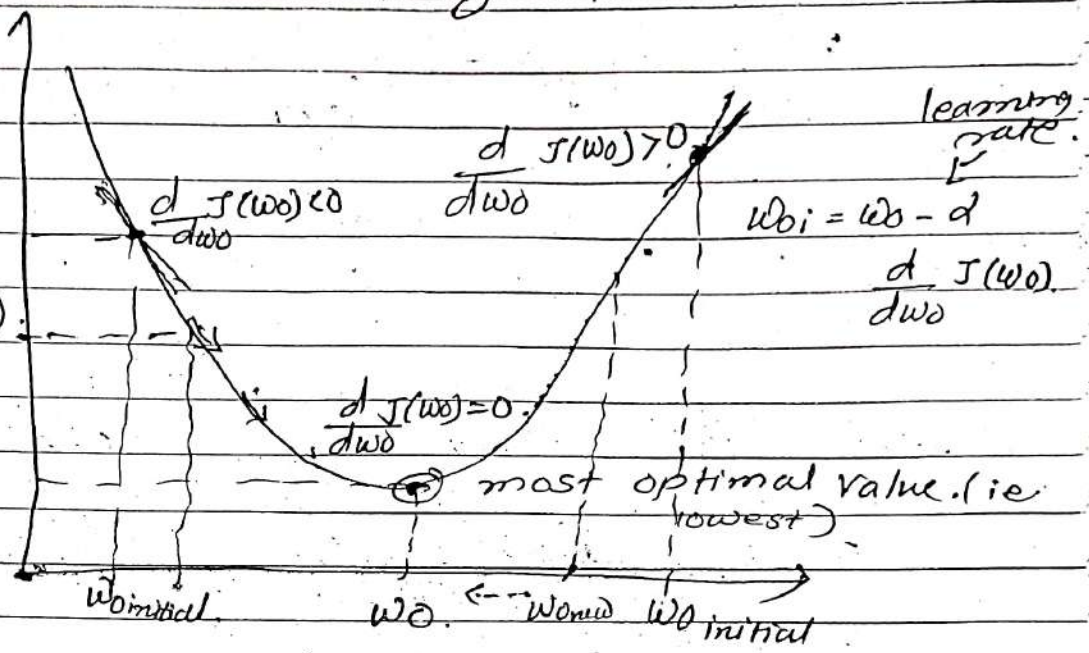
$$= \frac{1}{2N} \cdot \sum_{i=1}^{N} (\hat{y}_i - y_i)^2$$

this factor doesn't change anything but it simplies the later concept of calculus.

$J(W) = \cdots E$ If we find the value for w_0 & w_1 which minimizes this cost func', then we know that we get the best suited line.

$W \to (w_0, w_1)$ in our case.

Assume that, the cost func takes only 1. value w_0 to simplify it. $J(w_0)$.



If we select a first pt as $w_{0\,initial}$ the derivative of this pt is a tangent & its func is given by,

$\frac{d}{dw_0} J(w_0)$. (i.e the slope of a tangent)

then, the value of 'w_0' can be computed as,

$$w_{0i} = w_0 - \alpha \frac{d}{dw_0} J(w_0)$$

where α is a learning rate.

In the 1st w_0 which we selected,

$\frac{d}{dw_0} J(w_0)$ will be positive,

Therefore, if we subtract, ~~new values~~ w_0 will decrease.

This will continue till we reach the min. value of w_0.

~~$w_{0i} = w_0 - \alpha \frac{d}{dw_0} J(w_0)$~~

- This algo is called gradient descent

- If we select the initial w_0 at the left side of the curve, the value of

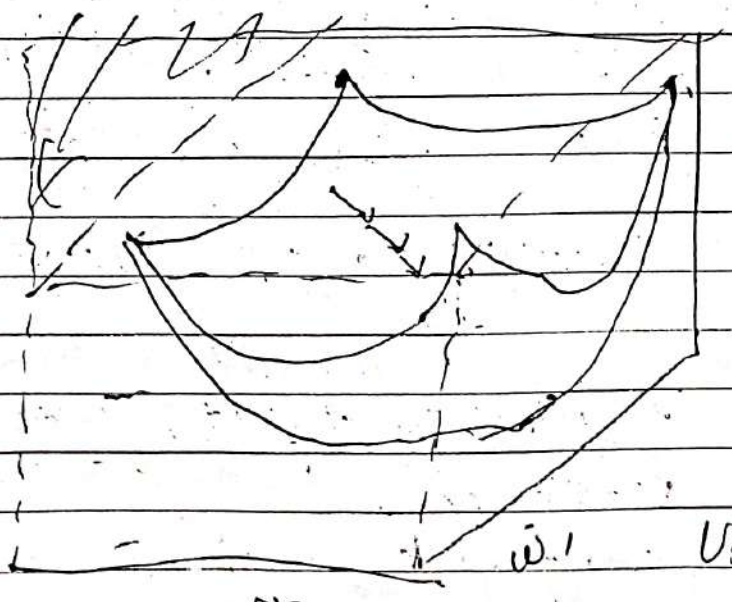
$$\frac{d}{dw_0} J(w_0) < 0$$

∴ This func will increase the value of w_0. So, in turn we will reach to the min value of w_0

Best case would be to take initial w_0 at the min.

i.e. $w_{0i} = w_0 - 0$ zero.

Coming back to the cost func of 2 par. $J(w_0, w_1)$, we have



We have surface for error.

To find $J(w_0, w_1)$.

Using gradient descent,

Repeat until convergence
{

$$w_j := w_j - \alpha \frac{d}{dw_j} J(w)$$

}

↑ simultaneously for every j.

- When w is a vector:

$x = \{x_1, x_2, \ldots, x_N\}^T$.

$y = \{y_1, y_2, \ldots y_N\}^T$.

$w = \{w_0, w_1, \ldots w_D\}^T$.

$$w_j = w_j - \alpha \frac{d}{dw_j} J(w).$$

x & y can also be a vector

i.e. $x_1 = \{x_{10}\ x_{11} \cdots x_{1D}\}$.

$x_2 = \{x_{20}\ x_{21}\ x_{22} \cdots x_{2D}\}$

w_0 = (2) to accomodate initial w_0 ; HP ; Acc ; wt. of vehicle etc.

In the previous ex, we had only MPG as an O/P par. (i.e. scalar par.) which depended upon HP, but we can also have no. of par. on which the O/P depends (i.e. x_1 & x_2 both can be vectors)

then the eq becomes,

$\hat{y}$.

$J(W) = \{ w_0, w_1 \cdots w_D \}$.

$X = \{ \underline{x}_1, \underline{x}_2 \cdots \underline{x}_N \}^T$.

$y = \{ \underline{y}_1, \underline{y}_2 \cdots \underline{y}_N \}^T$.

(1) $\hat{y}_i(x_i) = \sum_{j=0}^{D} x_{ij} w_j$ $\Leftarrow$ hypothesis. $w_0 + w_1 x_1 + w_2 x_2 + \cdots w_n x_n$

by defining $x_0 = 1$, we can write in column vector as, $= W^T x_i$

eg. $\hat{y}_i(x_1) = x_{10} \cdot w_0 + x_{11} w_1 + x_{12} w_2 + \cdots x_{1D} w_D$.

(2) $J(W) = \frac{1}{2N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2$ $\Leftarrow$ cost func^n.

(3) $w_j = w_j - \alpha \frac{d}{dw_j} J(W)$.

$$\frac{d}{dw_j} J(W) = \frac{1}{2N} \sum_{i=1}^{N} \frac{d}{dw_j} (\hat{y}_i - y_i)^2$$

$$= \frac{1}{2N} \sum_{i=1}^{N} 2 (\hat{y}_i - y_i) \cdot \frac{d}{dw_j} (\hat{y}_i - y_i)$$

$$= \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i) \cdot x_i$$

y_i doesn't depend upon w_j, so ignoring the term y_i in derivative.
Also $\hat{y}_i$ depends upon x_i from eq^n (1), so derivative is x_i

$\therefore$ (4) $w_j = w_j - \alpha \left[\frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i) \cdot x_i \right]$

Update rule.

eg → Given data

	x	y
0	1	1
1	2	3
2	4	3
3	3	2
4	5	5

$\alpha \Rightarrow$ learning rate
$\Rightarrow 0.01$

→ $\hat{y}_i(x_i) = w_0 + w_1 x_i$

$w_j = w_j - \alpha \left\{ \frac{d}{dw_j} J(w) \right\}$ lets call this as β.

lets take initial values of w_0 & w_1 as 0.

$\hat{y}_0(x_0) = 0 + 0 = 0.$

Error = $\beta = (\hat{y}_0 - y_0) = 0 - 1 = -1$

$w_0 = w_0 - (0.01) \cdot (-1).$
$= 0 - (0.01)(-1).$
$= 0.01.$

$w_1 = 0 - (0.01)(-1).$
$= 0.01.$

$\hat{y}_0(x_0) = 0.01 + 0.01\,(1).$
$= 0.01 + 0.01$
$= 0.02 //.$

$\beta = \text{Error} = 0.02 - 1 = -0.98$

$w_0 = 0.01 - (0.01) \cdot (-0.98)$
$= 0.0001$

$\hat{y}_1(x_1) = 0.01 + 0.01(2)$.

$= 0.03$

$\beta = (0.03 - 3) = -2.97$

$w0 = w0 - \alpha\beta$.

$= 0.01 - 0.01(-2.97)$

$\doteq 0.0397$

$w1 = w1 - \alpha\beta$.

$= 0.01 - 0.01(-2.97)$.

The process repeats till the predict^n is fairly near the actual value.

Refer London mlc learning study group

* Logistic Regression. (Supervised Algo).

- classificat^n deals with classifying the dataset into given class labels.
eg → tuple 'x' can be classified into "buys-computer" & "doesn't buy computer"

- Suppose we want to know the exact probability that the tuple 'x' will belong to class i.e. $P(C_i|x)$, then the concept of regression can be used for classification. where class labels are given as no.s. eg. 0 & 1 for binary regression, ~~then we~~

24th

- Binary logistic regression

I/P. $\mathcal{D}$ $X = \{x_1, x_2 \cdots x_N\}$
$y = \{y_1, y_2 \cdots y_N\}$ where $y_i \in \{0,1\}$

Here class labels y_i is either 0 or 1.

Goal is to model $P(y|x)$.

$P(y_i=1 | x_i, w) = \sum_j w_j x_{ij} = x_i w$

It has to be bet^n [0, 1]. — Any real value. — Any real value.

To compute the odds ⇒ $\frac{P_i \leftarrow \text{Prob. of selection}}{(1-P_i) \leftarrow \text{Prob. of not getting selected}}$

This value can ~~take~~ give any +ve no.

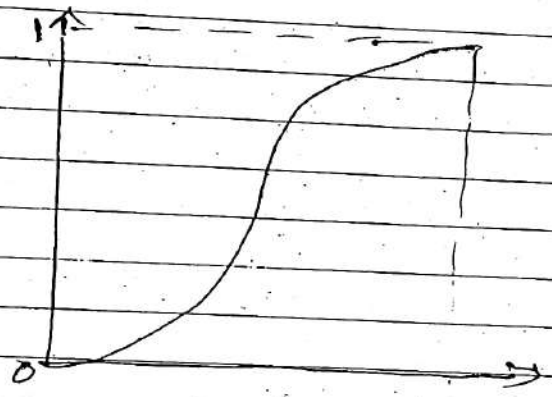
$$\frac{P_i}{1-P_i} = x_i w$$

Since this take any +ve value, lets take log of that.

$$\log\left(\frac{P_i}{1-P_i}\right) = x_i w \quad \leftarrow [0, 1]$$

Logistic Regression model.

$$\log\left(\frac{P_i}{1-P_i}\right) = \sum_{j=0}^{D} w_j x_{ij} = (x_i w) \quad \leftarrow \text{matrix notation}$$



standard logistic sigmoid function

~~The question is how to estimate model parameter w_j~~

Take inverse of eqn to get rid of log func.

$$\left(\frac{P_i}{1-P_i}\right) = e^{x_i w} \Rightarrow$$

Adding 1 on both sides

$$\frac{P_i}{(1-P_i)} + 1 = e^{x_i w} + 1$$

$$\frac{P_i + 1 - P_i}{(1-P_i)} = 1 + e^{x_i w}$$

$$\frac{1}{(1-P_i)} = 1 + e^{x_i w}$$

$$1 - P_i = \frac{1}{1 + e^{x_i w}}$$

$$\therefore P_i = \frac{e^{x_i w}}{1 + e^{x_i w}} = \frac{1}{1 + e^{-x_i w}}$$

$$P_i = 1 - \frac{1}{1 + e^{x_i w}}$$

$$= \frac{1 + e^{x_i w} - 1}{1 + e^{x_i w}}$$

$$\left[p_i = \frac{1}{1+e^{-x_i w}} \right]$$ Standard logistic sigmoidal func.

- Sigmoidal func always lies in 0 & 1.

Linear regression.

hypothesis $\Rightarrow \quad \hat{y}_i = x_i w$

logistic regression.

hypothesis $\Rightarrow \quad \hat{y}_i = \begin{cases} 1 & \text{if } \frac{1}{1+e^{-x_i w}} > 0.5 \\ 0 & \text{otherwise} \end{cases}$

$$\therefore \quad p(y_i = 1 \mid x_i, w) = \frac{1}{1+e^{-x_i w}} \quad \text{---(I)}$$

$$p(y_i = 0 \mid x_i, w) = 1 - \frac{1}{1+e^{-x_i w}} \quad \text{---(II)}$$

If we combine eqn (I) & (II)

$$\left[p(y_i \mid x_i, w) = \left(\frac{1}{1+e^{-x_i w}} \right)^{y_i} \cdot \left(1 - \frac{1}{1+e^{-x_i w}} \right)^{(1-y_i)} \right]$$

If $y_i = 1$, then the second term becomes '0'

If $y_i = 0$, then the 1st term becomes '0'

* To estimate model parameter 'w'.

- Maximum likelihood Estimation. (MLE)

$x = \{x_1, x_2, \ldots x_N\}^T$

$y = \{y_1, y_2 \ldots y_N\}$ where $y_i \in \{0, 1\}$.

$w = \{w_1, w_2 \ldots w_D\}$.

Step 1 - $p(y|x; w) = \prod_{i=1}^{N} \left(\frac{1}{1+e^{-x_i w}}\right)^{y_i} \cdot \left(1 - \frac{1}{1+e^{-x_i w}}\right)^{(1-y_i)}$

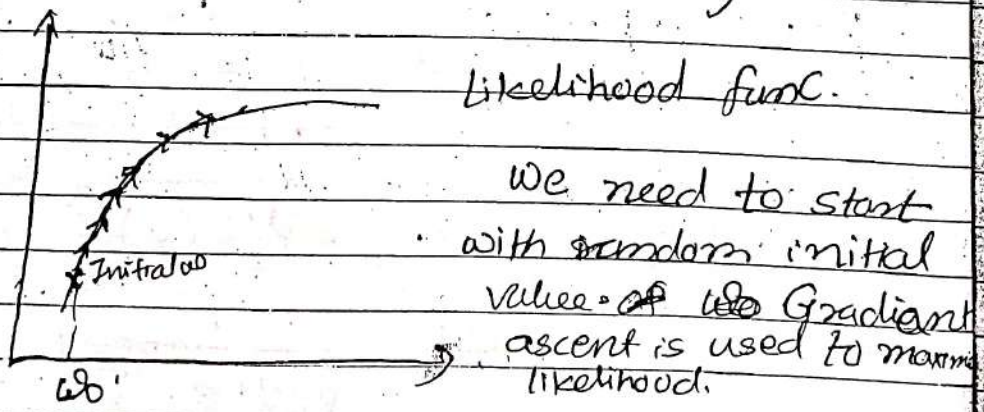
Step 2 - $L(w) = L(w; x, y)$.

likelihood func; func can vary; fixed parameters.

$L(w) = P(y|x; w)$.

Step 3 - Maximize $L(w)$.

$$L(w) = \prod_{i=1}^{N} \left(\frac{1}{1+e^{-x_i w}}\right)^{y_i} \left(1 - \frac{1}{1+e^{-x_i w}}\right)^{(1-y_i)}$$



But, since the likelihood func is difficult to differentiate we will take log of this func.

If we apply log rule of power (i.e. $\log a^b = a \log b$) of likelyhood func^c, it becomes

$$L(w) = \log L_1(w) = \sum_{i=1}^{N} \left[y_i \log \left(\frac{1}{1+e^{-x_i w}} \right) + (1-y_i) \log \left(1 - \frac{1}{1+e^{-x_i w}} \right) \right]$$

To compute the gradient of log likelihood func:

$$\nabla_w L(w) = \nabla_w \sum_{i=1}^{N} y_i \log(\sigma(x_i w)) + (1-y_i) \log(1-\sigma(x_i w))$$

$$= \sum_{i=1}^{N} y_i \frac{1}{\sigma(x_i w)} \cdot \frac{d(\sigma(x_i w))}{dw} + (1-y_i) \cdot \frac{1}{(1-\sigma(x_i w))} \cdot \frac{d}{dw}(1-\sigma(x_i w))$$

let $\sigma(x) = \frac{1}{1+e^{-x}}$ then,

$$\frac{d}{dx}\sigma(x) = \frac{d}{dx}\left(\frac{1}{1+e^{-x}}\right)$$

$$= \frac{e^{-x}}{(1+e^{-x})^2}$$

$$= \frac{1}{(1+e^{-x})} \cdot \frac{e^{-x}}{(1+e^{-x})}$$

$$= \frac{1}{(1+e^{-x})} \cdot \frac{(1+e^{-x})-1}{(1+e^{-x})}$$

$$= \frac{1}{(1+e^{-x})} \cdot \left(1 - \frac{1}{1+e^{-x}}\right)$$

$$= \sigma(x) \cdot (1-\sigma(x))$$

Similarly Taking derivative of log func.

According to previous derivation

$$= \sum_{i=1}^{N} y_i \frac{1}{\sigma(x_i w)} \cdot \sigma(x_i w)(1-\sigma(x_i w)) + (1-y_i) \cdot \frac{1}{1-\sigma(x_i w)} \cdot (-1)\sigma(x_i w)\cdot(1-\sigma(x_i w))\, x_i$$

$$= \sum_{i=1}^{N} y_i (1-\sigma(x_i w))\, x_i + (1-y_i)\cdot(-1)\cdot \sigma(x_i w)\cdot x_i$$

$$= \sum_{i=1}^{N} y_i x_i - y_i x_i \sigma(x_i w) - \sigma(x_i w) x_i + y_i x_i \sigma(x_i w)$$

$$= \sum_{i=1}^{N} y_i x_i - \sigma(x_i w) x_i$$

$$= \sum_{i=1}^{N} [y_i - \sigma(x_i w)]\, x_i$$

$$\nabla l(w) = \sum_{i=1}^{N} \left[y_i - \frac{1}{1+e^{-x_i w}} \right] x_i$$ ← Gradient of log likelihood func.

- Gradient ascent update rule.

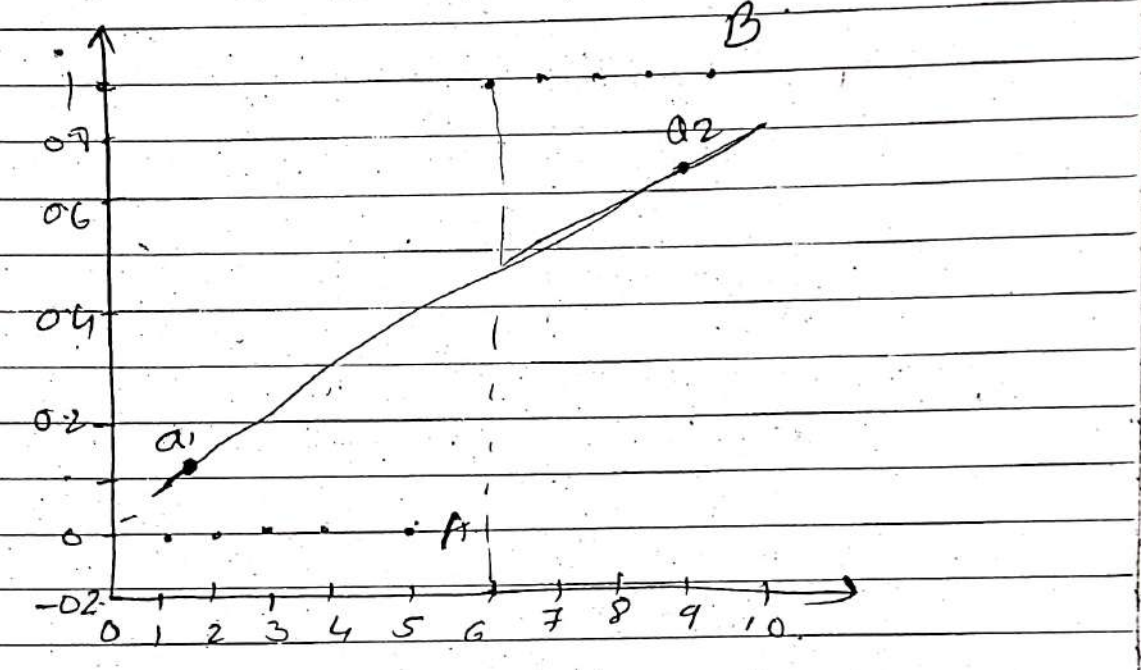
Repeat until convergence {

$$w_j = w_j + \alpha \sum_{i=1}^{N} \left(y_i - \frac{1}{1+e^{-x_i w}} \right) x_j$$

}

where α is a learning constant

eg → x = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
y → {0, 0, 0, 0, 0, 1, 1, 1, 1, 1}



If we consider pt- 'a1', the prob. is around 0.1 which suggest it might belong to class A.
If we consider pt. 'a2', the prob. is '0.7' which indicates the pt. may belong to class B.

*1) Hypothesis ⇒ $y_i = 1$ if $\frac{1}{1+e^{-x_i\omega}} > 0.5$

$= 0$ otherwise.

2) Likelihood fun^c ⇒

$$L(\omega) = \prod_{i=N}^{N} \left(\frac{1}{1+e^{-x_i\omega}}\right)^{y_i} \left(1 - \frac{1}{1+e^{-x_i\omega}}\right)^{(1-y_i)}$$

3) Update Rule -

$$\omega_j = \omega_j + \alpha \sum_{i=1}^{N} \left(y_i - \frac{1}{1+e^{-x_i\omega}}\right) x_j$$

☆ Decision Trees. (Refer to next page for notes) Ignore these notes

* A decision tree takes as i/p an object or situation described by a set of properties & outputs a yes/NO decision.

Eg → Decision - Whether to wait for a table at a restaurant.

Variables which affect our decision.

1- Alternate - Whether there is a suitable alternative restaurant

2 - Lounge - Whether there is a lounge for waiting customers

3- Fri/Sat - Whether it is friday or saturday (when there is rush)

4 - Hungry - Whether we are hungry

5 - Patrons - How many people are in it (none, some, full)

6 - Price - the restaurant's rating (*, **, ***)

7 - Raining - Whether it is raining outside

8 - Reservation - Whether we made a reservation

9 - Type - The kind of restaurant (Indian, chinese, Thai, Fast food)

Chapter 3

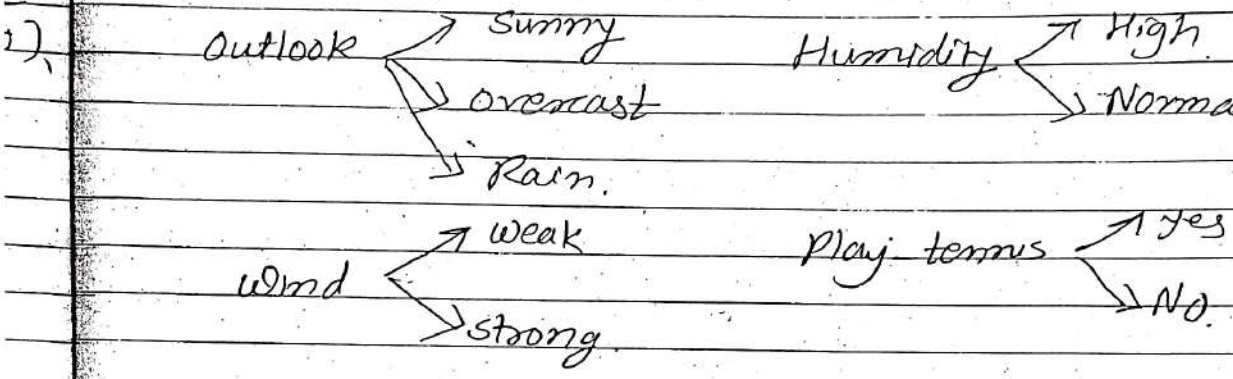
Learning with Trees

Refer victor lavrenko video notes.

Decision Trees:

- Its a supervised learning classifier.

Eg ⇒ Given a dataset for playing tennis.



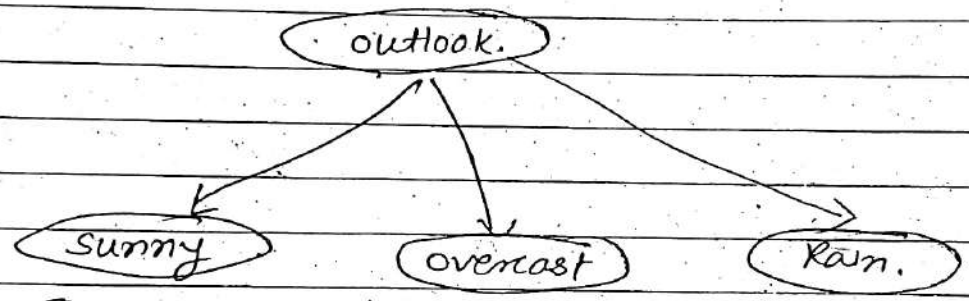
Day	Outlook	Humidity	Wind	Play
D1	S	H	W	N
2	S	H	S	N
3	O	H	W	Y
4	R	H	W	Y
5	R	N	W	Y
6	R	N	S	N
7	O	N	S	Y
8	S	H	W	N
9	S	N	W	Y
10	R	N	W	Y
11	S	N	S	Y
12	O	H	S	Y
13	O	N	W	Y
14	R	H	S	N

9 yes / 5 No

tupple to classify ⇒ { Rain, High, Weak }

- To construct a decision tree.
  - Divide & conquer.
    - Split into subsets.
    - are they pure ? (i.e. all yes or all NO).
    - If yes : stop
    - If NO : repeat.

Lets take outlook as our 1st attr.



Sunny: 5 tuples ⇒ yes · 3 — 11 → NO.

overcast: 4 tuples ⇒ yes. (pure subset)

Rain: 3 ⇒ yes. 2 ⇒ NO

D1	S	H	W ·
D2	S	H	S ·
D8	S	H	W ·
D9	S	N	W ⇒ yes ·
D11	S	N	S · ⇒ yes.

(Split further)

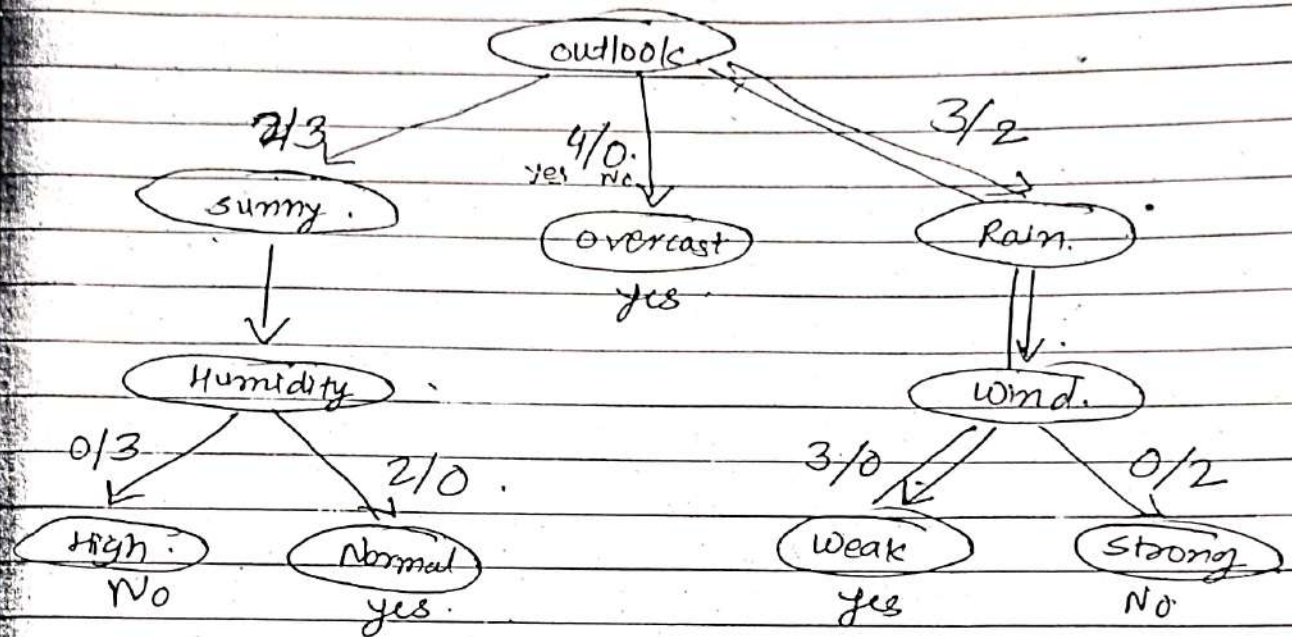
If we split on humidity, we can get a pure subset.

D4	R	H	W ⇒ yes
D5	R	N	W ⇒ yes
D6.	R	N	S.
D10	R	N	W ⇒ yes
D14	R	H	S ·

(Split further)
Here if we split on windy, we can get pure subset.

Dividing based on this criteria,

9yes / 5 No.



To classify (Rain, High, Weak) ⇒ John plays.

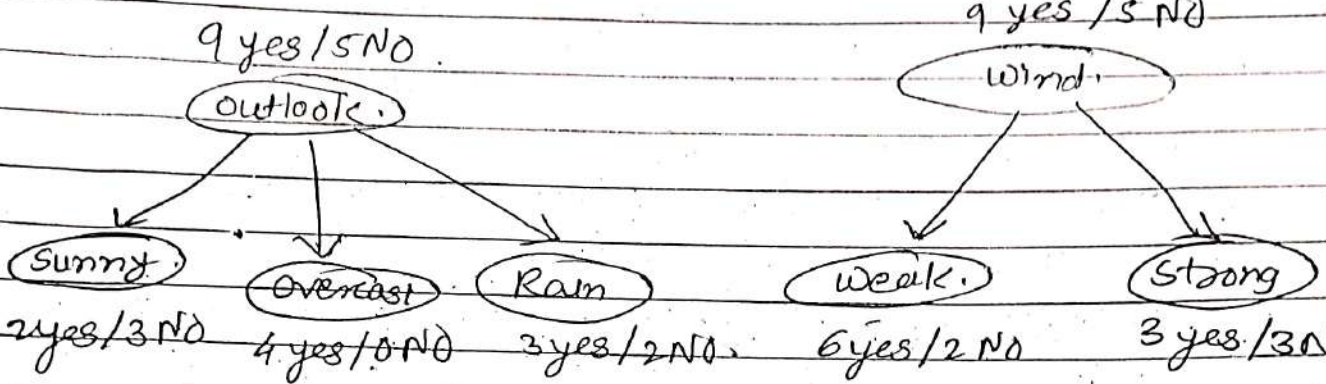
The decision trees not only classify the tuples, but also give you confidence by which we classify (eg → 9/5, 4/0 etc). The more the no. of dataset, the more will be the confidence).

Note ⇒ Always try to take the 1st attr which gives you a pure subset.

* ID3 Algorithm. (Iterative dichotomiser 3).

Split (node, {examples}) :
1- A ← the best attr for splitting.
2- Decision attr for this node ← A.
3- for each value of A, create a child node.
4- Split training {examples} to child nodes.
5- For each child node / subset :
if subset is pure : STOP.
else : split (child_node, {subset})

* Which Attr to split on ?



1 = pure set 0 ⇒ pure set.

* Measure the "purity" of the split :

- Certainity about yes/No after split.
  - pure set (4 yes / 0 NO) ⇒ completely certain (100%)
  - impure (3 yes / 3 NO) ⇒ completely uncertain (50%)
- Can't use P ("yes" | set) :
  - must be symmetric :
    4 yes / 0 No as pure as 0 yes / 4 No

* Entropy - (way to measure uncertainty)

$$H(S) = -P_{(+)} \log_2 P_{(+)} - P_{(-)} \log_2 P_{(-)} \text{ bits}$$

where S ⇒ subsets of training ex.

$P_{(+)}$ / $P_{(-)}$ ⇒ % of positive / -ve examples in S.

Interpretation ⇒ Assume 'x' belongs to 'S'
- how many bits need to tell if x is +ve or -ve

Ex ⇒ Impure (3 yes / 3 No).

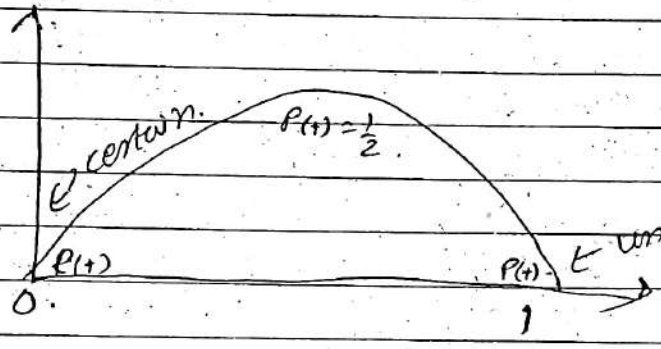
$$H(S) = -\frac{3}{6}\log_2\frac{3}{6} - \frac{3}{6}\log_2\frac{3}{6} =$$

$$= 1 \text{ bits.}$$

∞ Pure set (4 yes / 0 No).

$$H(S) = -\frac{4}{4}\log_2\frac{4}{4} - \frac{0}{4}\log_2\frac{0}{4}$$

$$= 0 \text{ bits.}$$



* Information Gain.

- Want many items in pure sets.
- Expected drop in entropy after split -

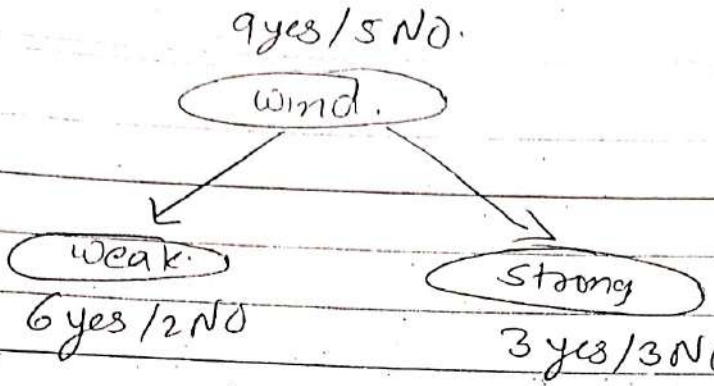
$$Gain(S, A) = H(S) - \sum_{v \in values(A)} \frac{|S_v|}{|S|} H(S_v).$$

where, V ⇒ possible values of A
S ⇒ set of examples {x}.
S_v ⇒ subset where $x_A = V$.

- Mutual Informat^n.

- bet^n att^x A & class labels of S.

eg →



$$H(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.94$$

$$H(S_{weak}) = -\frac{6}{8}\cdot\log_2\frac{6}{8} - \frac{2}{8}\log_2\frac{2}{8} = 0.81$$

$$H(S_{strong}) = -\frac{3}{6}\log_2\frac{3}{6} - \frac{3}{6}\log_2\frac{3}{6} = 1.$$

Gain (S, Wind) =

$$H(S) - \frac{8}{14}H(S_{weak}) - \frac{6}{14}H(S_{strong})$$

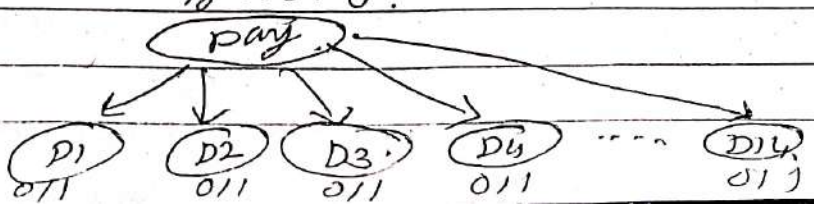
$$= 0.94 - \frac{8}{14} * 0.81 - \frac{6}{14} * 1$$

$$= 0.049$$

- The gain for each attr in the given dataset is computed. The Attr which gives max gain is selected for splitting criteria.

* Problems with information Gain

Eg → If we split on attr Day,



All subsets are perfectly pure

- Here, the data will not be repeated i.e. next day to come will be D15 & so on.
- Info. gain is biased towards attr. with many values.
- It won't work for new data.

∴ Use Gain Ratio: (Used by C4.5 Algo).

$$\text{SplitEntropy}(S,A) = -\sum_{V \in \text{values}(A)} \frac{|S_V|}{|S_A|} \log \frac{|S_V|}{|S|}$$

where, A ⇒ candidate attr.
V ⇒ possible values of A.
S ⇒ set of examples ∀ y
S_V ⇒ subset where $x_A = V$.

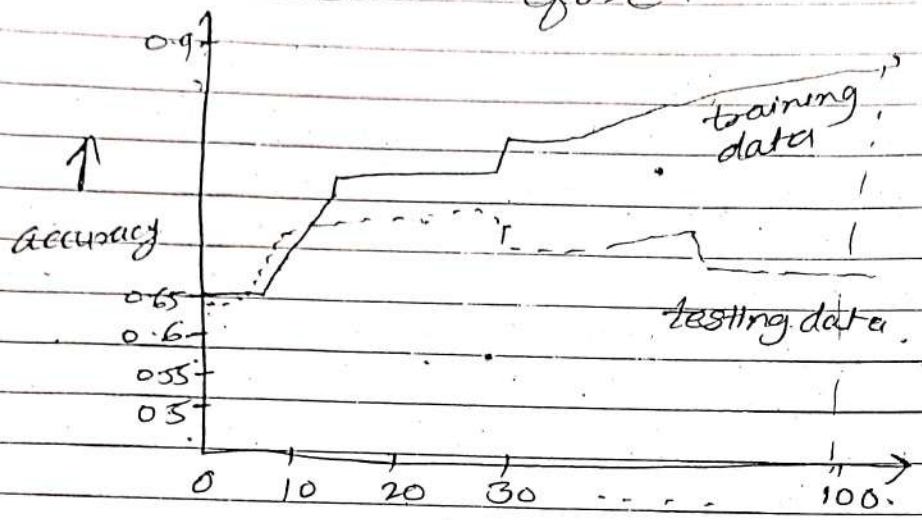
$$\text{GainRatio}(S,A) = \frac{\text{Gain}(S,A)}{\text{SplitEntropy}(S,A)}$$

* Overfitting in Decision Trees.

- DT classifies training eg. perfectly
  - Keep splitting until each node has 1 example
    - Singleton = pure.
- It doesn't work for new data.

∴ As the tree keeps splitting the data, it gets bigger & bigger & more accurate

- But that means it becomes less accurate on the data it has not seen before.



- At the end the size of tree will be same as size of dataset.
- Referring to the graph of accuracy, as the size of tree increases, the testing data accuracy decreases.

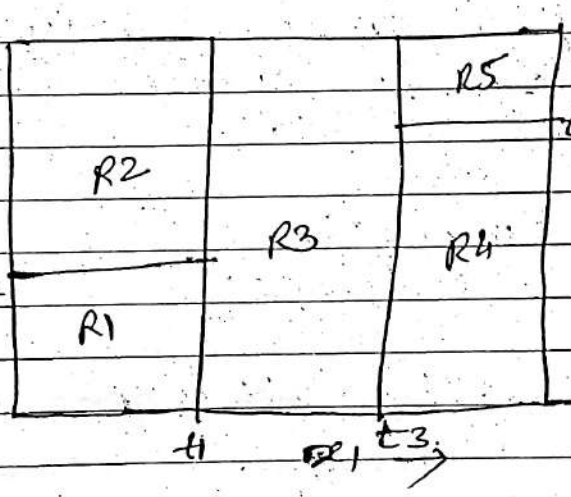
- How to deal with this?
- Avoid overfitting.
  - Stop splitting when not statistically significant.
- Grow then post-prune.
  - based on validatn set. (i.e. testing data)
- Sub-tree replacement pruning

  For each node.
  - Pretend remove node + all children from tree.

- Measure performance on validatn set (ie. testing data).
- Remove the node that results in greatest improvement
- Repeat until pruning is harmful.

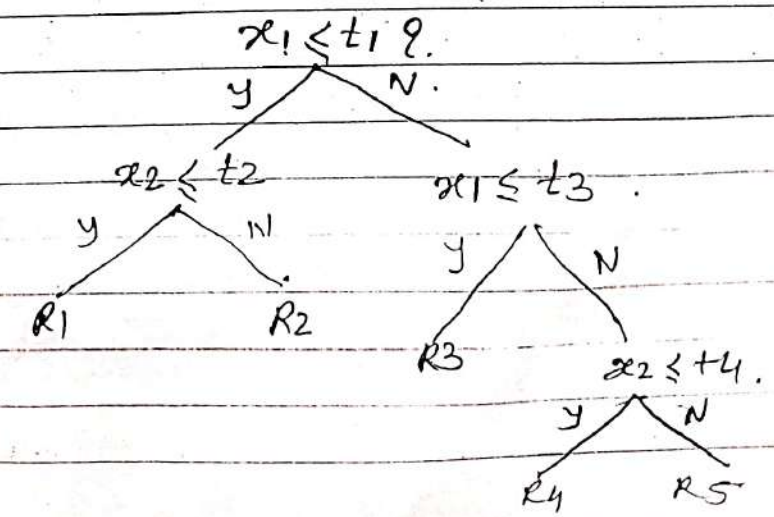
* Classification & Regression Trees (CART)

- Partition the I/P space into rectangle.
- Assume 2D I/P space.



- Divide this into rectangles by using axes ||el lines.

Construct^n of decision trees using the rectang.



- Tree gives only two branches at every point. i.e. its a binary trees.
- It uses divide & conquer approach;

- Depending upon the problem we are solving (i.e. classificatn & regression) we select a single value from each region.

- If it is a regression problem,

O/P is a single value for entire region. (eg. if your o/p is falling in region, R1, regardless of anywhere in R1 it falls, the algo. return same value).

i.e. regression gives same real valued o/p for each region.

- classificatn problem.

regardless of anywhere in a region the o/p falls, the algo. will return same class label.

i.e. classificatn gives same class label for entire region.

* Regression trees

The goal of regression is to minimize error.

i.e. fit a func to minimize

$$\leq (y_i - f(x_i))^2$$

$$f(x) = \sum_{m=1}^{M} c_m I(x \in R_m)$$

Suppose, the tree has split I/P space into M regions. (i.e. R_1 to $\cdots R_M$). For each region a specific value is given as O/P. Lets denote it by c_m.

I is an indicator func i.e. shows whether data pt 'x' lies in region R_m or not.

$f(x)$ will be non zero for only 1 term where the term 'x' falls into some region. Rest others, $f(x)$ will give value 0.

i.e. $I(x \in R_m) = 1$

$I(x \notin R_m) = 0$.

- How to select best value for c_m to give as an O/P for particular region 'm'.

$$\text{Best } \hat{c}_m = \text{avg}(y_i \mid x_i \in R_m)$$

i.e. average of all 'y' values for the selected region.

This means that, given a region split, what is the best possible O/P that can be predicted.

$$\sum (y_i - f(x_i))^2$$

$$f(x) = \sum_{m=1}^{M} c_m I(x \in R_m)$$

Suppose, the tree has split I/P space into M regions. (i.e. R_1 to $\cdots R_M$). For each region a specific value is given as O/P. Lets denote it by c_m.

I is an indicator func i.e. shows whether data pt 'x' lies in region R_m or not.

$f(x)$ will be non zero for only 1 term where the term 'x' falls into some region. Rest others, $f(x)$ will give value 0.

i.e. $I(x \in R_m) = 1$

$I(x \notin R_m) = 0$

→ How to select best value for c_m to give as an O/P for particular region 9.

$$\text{Best } \hat{c}_m = \text{avg}(y_i \mid x_i \in R_m)$$

i.e. average of all 'y' values for the selected region.

This means that, given a region split what is the best possible O/P that can be predicted.

* How to find regions ? (i.e. best R_m's)

- Greedy approach for finding region.

- Considering split variable.
(Try to find best split pt.).

$R_1(j,s) = \{x \mid x_j \leq s\}$.

$R_2(j,s) = \{x \mid x_j > s\}$.

subregion where.

R_1 is that, j^{th} coordinate of variable 'x' is less than some chosen value 's'.

The problem is to find 'j' & 's'.

$$\left[\min_{c_1} \sum_{x_i \in R_1(j,s)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j,s)} (y_i - c_2)^2 \right]$$

squared error for all data pt. lying in region R_1

squared error for all data pt. lying in region R_2

~~Similarly~~ find 'j' & 's' which minimize this entire expression.

∴ the dataset is finite, there will be only finite 's' that have to be tried for every 'j'

- One the best (j, s) is found, split the data into 2 parts.
- Repeat the process for all regions.

- Where to stop ? in creating the regions

- Grow the tree to the pt. where you have only few data pts. in leaves.

or

- Grow a large tree & then prune

* Classification Trees.

$$\hat{P}_{mk} = \frac{1}{N_m} \sum_{x_i \in R_m} I\{y_i, k\}$$

No. of data pts. in region 'm' that belongs to class 'k'

$\hat{P}_{mk}$ ⇒ prob. that data pt. 'm' belongs to class 'k' in region 'm'

$I(y_i, k)$ ⇒ Indicator fun^c that 'y$_i$' belongs to class 'k'

N_m ⇒ Total no. of pts. in region 'm'

To return class label.,

$$\text{class}(m) = \underset{k}{\text{argmax}}\ \hat{P}_{mk}$$

Error measures. -

Misclassificat^n. error

$$\frac{1}{N_m} \sum_{i \in R_m} I(y_i \neq \text{class}(m)).$$

$$= 1 - \hat{P}_{m\,\text{class}(m)}$$

Cross Entropy or Deviatn.

$$= -\sum_{K=1}^{K} \hat{p}_{mk} \cdot \log \hat{p}_{mk}$$

Gini Index $= \sum_{K=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$.

These two measures lead to more accurate tree creation.

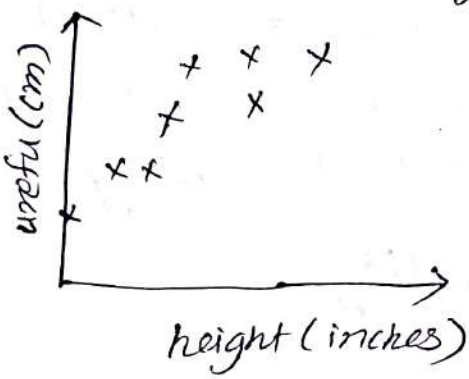
Chapter 6

Dimensionality Reduction

* Dimentionality Reduction.

Refer victor lavrenko

- True Vs. Observed dimensionality.
  - suppose we have a dataset where instances are represented as attr pair ℰ urefu, height 𝒥.
  - If we try to figure out what is the dimensionality of this data, by observatn it looks like 2D data.



  - But if search the meaning of 'urefu', it actually is an african word, which means height only
  - So, it actually is 1D data but since one height is measured in 'cms' & another in 'inches', we get a 2D plot
  - In this example, the true dimensionality of a problem is different (i.e. less) than the observed dimensionality.

- Another Example.
  - Datapts. collected from diff. geographic areas. over time.
    - X_1 ⇒ No. of skidding accidents.
    - X_2 ⇒ —11— burst of water pipes
    - X_3 ⇒ snow-plow expenditure.
    - X_4 ⇒ No. of school closure.
    - X_5 ⇒ No. of patients with heat stroke
  - Here, the data looks like a 5D data. but if we observe closely, all of these attr are related to only one specific attr i.e. temperature. which takes the values as < freezing pt, normal & very hot. That means the real dimension of this problem is actually '1D'

- Why the dimensionality is a problem?.
  - Most of the datasets like images, speech, text are high dimensional. But all these dimensionalities are not factual. This means that while dealing with these datasets, we choose to select seperate attr for these datasets. (i.e. we select a seperate attr for each word in case of text & we select to select each pixel of an image as attr)., most of which can be just noise which can be ignored or is not playing a part in m/c learning.

- Dealing with high Dimentionality.
  - Use domain knowledge:
    - Use feature extraction techniques such as (SIFT) (used in CV), MFCC
    - It requires detailed domain knowledge to extract these features
  - Make Assumptions about the dimensions.
    - Independence - count along each dimension seperately
      eg → given 2D dataset having attr x_1 & x_2, then count x_1 ignoring the value of x_2. i.e. we are counting value of x_1 for each value of x_2.
    - Smoothness - Propagate class counts to neighboring regions. (this approach is used by KNN)
    - Symmetry - Invariance to order of dimensions. (i.e. Flipping x_1 & x_2 without changing their values). The observatns that are made for attr x_1 are now made for x_2

- Reduce the dimensionality of data

- create a new dimensions. where the goal is to represent the instances with fewer variables. while trying to preserve as much structure in the data as possible.

- Feature selection.

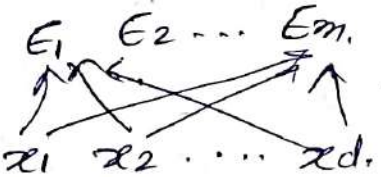
- Pick up a subset of original dimensions $x_1, x_2, x_3 \dots x_{d-1}, x_d$.

- We can use info. gain to throw away some attr if the goal is classification.

- Feature Extraction.

- Build entirely new set of variables to represent the data. (i.e. construct a new set of dimensions.

$$E_i = f(x_1, x_2 .. x_d).$$



- Each new attr is some combinatn or some func of original attr.

* Principal Component Analysis. (PCA).

- PCA problems are easily solvable using ANN. Thats why PCA ~~are~~ is popular dimensionality reduct^n technique.
- We have seen that SVM uses the concept of increasing dimensionality to make any linearly non separable data to linearly separable one. But, since increasing the dimensionality of a data, makes the computat^n complex & slower, PCA deals with dimensionality reduction.
- To do this, we need to find out whether in the given i/p data, any redundancy is present
- If the i/p is 'm' dimensional, we need to find out if there is any redundancy in this data whereby we can keep 'mo' no. of inputs & discard the remaining 'm-mo' no. of inputs where mo < m.
- This is possible only if the data which we are discarding is really insignificant.
- i.e. Given a vector 'x' of 'm' dimensions,

$\vec{x} = \{ x_1, x_2, x_3 \ldots\ldots x_{mo}, x_{mo+1} \cdots x_m \}$.

where m is very large no.
Here, if we decide to keep the data upto 'mo' dimension & discard the remaining data,

$\vec{x} = \{ x_1, x_2, x_3, \ldots x_{mo} \}$ ~~$x_{mo+1} \cdots x_m$~~

- We can do this only if we are sure that $(x_m - x_{mo})$ is a really insignificant data.
- This means that, if we are simply chopping off the data by making all the terms of $(x_m - x_{mo})$ to zero, we are introducing error.

- This error is given by.

  MSE = sum of variances of the elements which are eliminated.

- We can tolerate this error iff sum of variances is very small.

- PCA deals with this (i.e. which att's to select) using the following.

- It defines a set of principal components. as follows.

  1) The direction of greatest variability in the data.

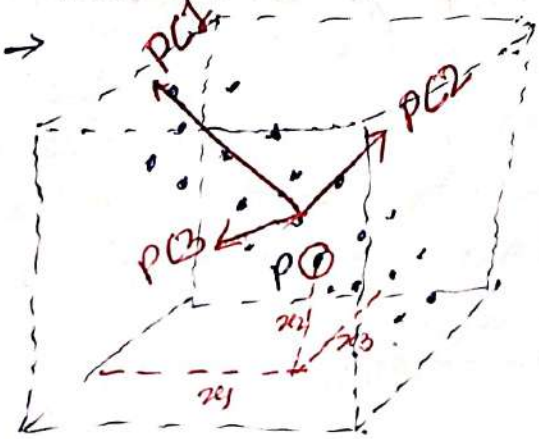
     - It checks for the dimensions of greatest variance. i.e. it checks for the dimension along which the data is spread out as much as possible.

  2) Perpendicular component to the 1st component, greatest variability of what's left.

  3) Step 2 repeats until the original dimensionality is covered.

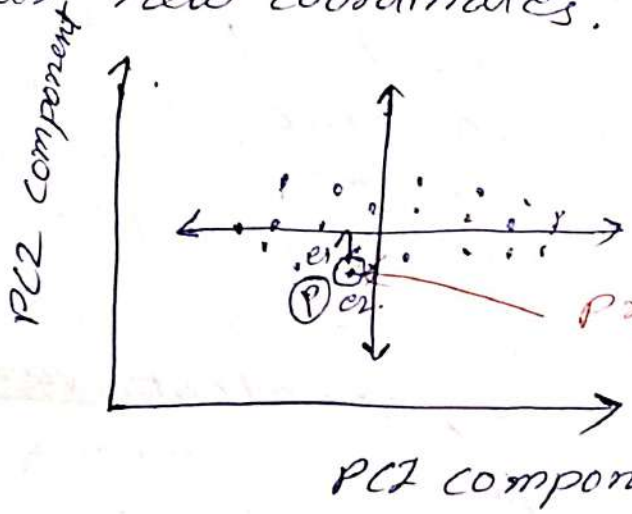
- From the given dataset of 'd' dimensions, the first 'm' < d components become 'm' new dimensions.

eg →



- If we consider this '3D' dataset & observe the datapts, most of the datapts are spread across only 1 dimension. So, that becomes our PC1.

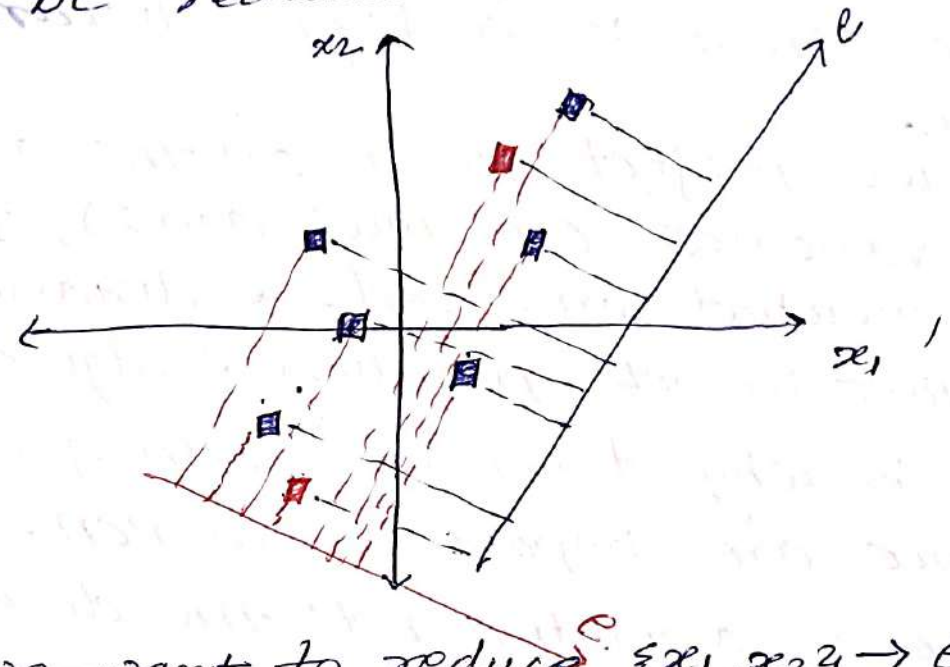
- The next principal component will be the second dimension where the data is spread & far to the PC1.
- Next principal component will be in the 3rd dimension. far to both PC1 & PC2
- Here, since all the datapts are spread on a single sheet, we would pick ~~the~~ only two PC1 & PC2 as our new coordinate vectors. & represent all the pts. ~~as~~ w.r.t. our new coordinates.



- Each pt. is taken & them projected on the axes PC1 & PC2. For the pt. 'P' we are getting slight -ve value for PC1. & highly -ve ~~valy~~ value for secord component PC2.
- Here, we actually went from pt. in '3D' to pt. in '2D'.

- Why should we consider greatest variance? (4).

Ex - Consider a given '2D' dataset which has to be reduced to '1D'

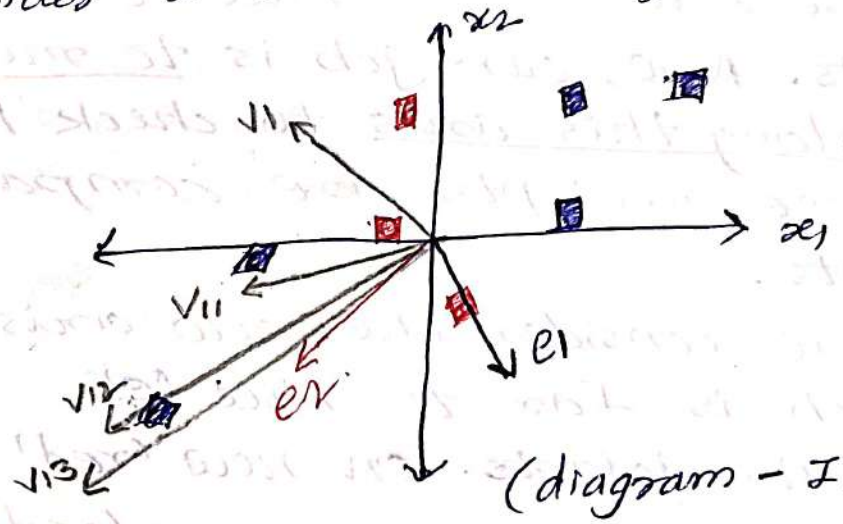


- Here we want to reduce $\{x_1, x_2\} \rightarrow e'$ (along new axis e).
- let us consider the new axis in blue as 'e' where the data is spread out more.
- Project all the datapts. in that direction of 'e'. All these new pts. on blue 'e' are the new datapts. Now, our job is to measure variance along this axis to check how varied these new pts are compared to the old pts.
- Let us now consider the new axis in red as 'e' which is far to blue 'e'.
- Project all the datapts. on new 'red' axis & measure the variance on new 'red' axis.
- We can conclude that the variance along 'blue' dimension is higher as compared to variance along 'red' dimension.
- Why it is important to select axis of high variance?

- Consider the two red pts. If we project them on 'red' axis, they land almost on top of each other but in original (x,y) space, there is a large distance betn them.
- If we project them on the axis having high variance (i.e. blue axis), the distance is somewhat preserved. & distances are important in ML application & algorithms.
- This is why dimensions with greatest variance are important in PCA.
- There is a relation betn the dimensions of highest variance & covariance of the data..

* Covariance Eigenvectors.

Consider a '2D' dataset given below.



(diagram - I)

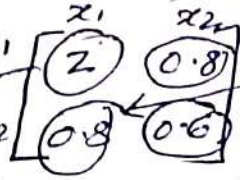
- Covariance -

- Move the dataset so that the center is at '0' (zero). To do this, we need to subtract the mean value of that attr from every instance. (for every attr)

$[x_{i,a} = x_{i,a} - \mu_a]$ This makes the math a lot simpler.

- Compute covariance matrix 'Σ'

- For 2 dimensions, we are going to get 2×2 matrix. (eg. x_1 | x_1: 2, x_2: 0.8; x_2: 0.8, 0.6)



covariance along 1st & 2nd attr.

spread of a data along first attr x_1.

spread of data along x_2 attr.

- covariance is given by,

$$\left[cov(x_1, x_2) = \frac{1}{n} \sum_{i=1}^{n} x_{i,1} \cdot x_{i,2} \right]$$

- Generally, covariance is computed by subtracting means, but here since we have already centered the data, the mean is not subtracted. (since now all the means are zero).

- Way to interpret the covariance is how one attr changes as the other changes. (i.e. if we increase / decrease x_1, how x_2 is going to increase / decrease).

- The pts. in blue in the graph represent +ve covariance i.e. x_1 & x_2 are varying in same direction.

- The pts. in red represent -ve covariance. i.e. x_1 & x_2 are not varying in same direct^n. (i.e. x_1 increasing in -ve direct^n, x_2 increasing in +ve direct^n.

- Similarly, we can compute variance for x_2 as

$$var(x_2) = \frac{1}{n} \sum_{i=1}^{n} x_{i,2}^2$$

- It is observed that the covariance matrix does interesting things when we multiply any random vector with this matrix.

eg → suppose we take any random vector [-1, 1] & multiply it with our covariance matrix

$$\begin{bmatrix} 2 & 0.8 \\ 0.8 & 0.6 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1.2 \\ -0.2 \end{bmatrix}$$

- [-1, 1] is a vector V_1 shown in diagram '1' in the second quadrant.

- If we plot the resultant vector [-1.2, -0.2], it turns out to be in the 3rd quadrant. (represented as V_{11}) in figure 1.

- If we multiply the resultant vector [-1.2, -0.2] with the covariance matrix

$$\begin{bmatrix} 2 & 0.8 \\ 0.8 & 0.6 \end{bmatrix} \begin{bmatrix} -1.2 \\ -0.2 \end{bmatrix} = \begin{bmatrix} -2.5 \\ -1 \end{bmatrix}$$

- If we plot this resultant vector in figure 1 (represented as V_{12}), we can observe that this vector is slightly longer.

- As we multiply the resultant vectors with covariance matrix, we can observe that the o/p vectors are rotating in clockwise direction & turning longer.

- If we repeat the process,

$$\begin{bmatrix} 2 & 0.8 \\ 0.8 & 0.6 \end{bmatrix} \begin{bmatrix} -2.5 \\ -1 \end{bmatrix} = \begin{bmatrix} -6 \\ -2.7 \end{bmatrix}$$

& plot this vector (represented as V_{13}), it is observed that the resultant is getting longer but now its turning by smaller & smaller angle.

- If we repeat the process further the resultant comes out to be.

$$\begin{bmatrix} -14.1 \\ -6.4 \end{bmatrix} \rightarrow \begin{bmatrix} -33.3 \\ -15.1 \end{bmatrix}$$

- If we compute the slope of these resultant vector, by dividing 2nd component by 1st component

$$\begin{bmatrix} -2.5 \\ -1 \end{bmatrix}, \begin{bmatrix} -6 \\ -2.7 \end{bmatrix}, \begin{bmatrix} -14.1 \\ -6.4 \end{bmatrix}, \begin{bmatrix} -33.3 \\ -15.1 \end{bmatrix}$$

Slope = 0.4 0.450 0.454 0.454.

- We can observe that the slope is getting converged at some value. (i.e. 0.454).
- If we observe the figure, the vector belonging to this slope happens to lie in the direction of vector 'e2'. This is the direction of greatest variance i.e. this is the direction where the data is highly spread.
- This proves that, as we multiply a vector using covariance matrix, it seems to be turning towards the direction where the data is spread out maximally & as we keep multiplying the vector with covariance matrix, at some point, the vector will stop changing the directn & it will converge where the directn of vector would be same as direction of highest covariance.
- Such vectors are called as Eigen vectors. These vectors have the property that if we multiply them with covariance matrix, they dont change. They only get multiplied with a scalar vector. (i.e. they become longer or shorter but they dont turn).

$$[\Sigma e = \lambda e.]$$

where $\Sigma \Rightarrow$ covariance matrix
$e \Rightarrow$ Eigen vectors.
$\lambda \Rightarrow$ scalar vectors (also known as Eigen values)

- Principal components in PCA
  - Eigenvector with the largest eigenvalues.

* Finding Eigenvectors.

1) Find Eigenvalues by solving

$$[\det(\Sigma - \lambda I) = 0.]$$

eg → $$\det\begin{pmatrix} 2-\lambda & 0.8 \\ 0.8 & 0.6-\lambda \end{pmatrix} = 0.$$

$$\therefore (2-\lambda)(0.6-\lambda) - (0.8)(0.8) = 0.$$

$$\therefore \lambda^2 - 2.6\lambda + 0.56 = 0.$$

If we solve this, $$[\lambda_1 = 2.36,\ \lambda_2 = 0.23]$$

2) And ith eigenvector by solving.

$$[\Sigma e_i = \lambda_i e_i]$$

eg → $$\begin{bmatrix} 2 & 0.8 \\ 0.8 & 0.6 \end{bmatrix}\begin{bmatrix} e_{1,1} \\ e_{1,2} \end{bmatrix} = 2.36\begin{bmatrix} e_{1,1} \\ e_{1,2} \end{bmatrix}$$

This gives,

$$2e_{11} + 0.8e_{12} = 2.36e_{11} \quad \text{②}.$$

$$0.8e_{11} + 0.6e_{12} = 2.36e_{12}$$

If we solve these two eqns,

$$e_{1,1} = 2.2\, e_{1,2}. \Rightarrow e_1 \sim \begin{bmatrix} 2.2 \\ 1 \end{bmatrix}$$

Any multiple of this vector 'e1' like (4.4,2) would work the same. So, there will be infinite no. of eigenvectors which can satisfy this eqn. We are going to pick any one randomly.

- We are going to make e1 of unit length (the reason will be covered later)

$$[\|e_1\| = 1]$$

- To make the vector of unit length, we need to divide it by its euclidean length. then,

$$e_1 = \begin{bmatrix} (2)/\sqrt{2.2^2+1^2} \\ (1)/\sqrt{2.2^2+1^2} \end{bmatrix} = \begin{bmatrix} 0.91 \\ 0.41 \end{bmatrix}$$

slope: 0.454.

Similarly,

$$\begin{bmatrix} 2 & 0.8 \\ 0.8 & 0.6 \end{bmatrix} \begin{bmatrix} e_{21} \\ e_{22} \end{bmatrix} = 0.23 \begin{bmatrix} e_{21} \\ e_{22} \end{bmatrix}$$

If we solve this, we would get,

$$e_2 = \begin{bmatrix} -0.41 \\ 0.91 \end{bmatrix}$$

The trick to solve this is, Since 'e2' is tar to e_1 it would have the reversed value with 1st (-) sign.

3) 1st PC $= \begin{bmatrix} 0.91 \\ 0.41 \end{bmatrix}$, 2nd PC $= \begin{bmatrix} -0.41 \\ 0.91 \end{bmatrix}$

★ Projecting the data to new dimensions.

- Suppose we have a 'd' dimensional dataset & we have identified 'm' principal components.
- $e_1, e_2 \cdots e_m$ are the new dimension vectors.
- We want to project the 'd' dimensional instances $x = \{x_1, x_2 \cdots x_d\}$ to the new dimensions. 'm'.
- Procedure to get new coordinates $x' = \{x_1', x_2' \cdots x_m'\}$.

step 1 ⇒ "Center" the instance by subtracting the mean: $(x' - u)$.

step 2 ⇒ Project each pt on new dimension. by taking dot product e_j $[(x' - u)^T e_j$ for $j = 1, 2 \cdots m]$

i.e.

$$(\vec{x} - \vec{u}) = [(x_1 - u_1), (x_2 - u_2) \ldots (x_d - u_d)]$$

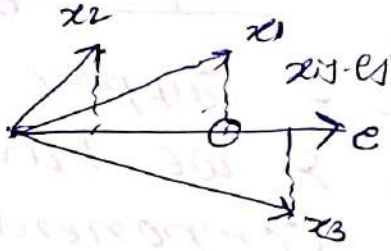
$$\begin{bmatrix} x_1' \\ x_2' \\ \vdots \\ x_m' \end{bmatrix} = \begin{bmatrix} (\vec{x} - \vec{u})^T \vec{e_1} \\ (\vec{x} - \vec{u})^T \vec{e_2} \\ \vdots \\ (\vec{x} - \vec{u})^T \vec{e_m} \end{bmatrix} = \begin{bmatrix} (x_1 - u_1) e_{11} + (x_2 - u_2) e_{12} + \cdots + (x_d - u_d) e_{1d} \\ (x_1 - u_1) e_{21} + (x_2 - u_2) e_{22} + \cdots + (x_d - u_d) e_{2d} \\ \vdots \\ (x_1 - u_1) e_{m1} + (x_2 - u_2) e_{m2} + \cdots + (x_d - u_d) e_{md} \end{bmatrix}$$

* Proof of Eigenvectors are actually pointing in the direction of greatest variability.

- Given the datapts (x_1, x_2, x_3), select vector 'e' s.t. when we project these pts on vector 'e', the spread of these pts would be as high as possible.
- Suppose we have a certain vector 'e', what is the variance along that vector 'e'? The variance is given by,

$$\frac{1}{n} \sum_{i=1}^{n} \left(\sum_{j=1}^{d} x_{ij} e_j - u \right)^2$$

This dot product is the projectn of vector x_i on the dimension e_j



- Assume, the mean is zero, then the variance becomes,

$$\frac{1}{n} \sum_{i=1}^{n} \left(\sum_{j=1}^{d} x_{ij} e_j \right)^2$$

- Now, we are looking at the vector 'e' which makes this variance as large as possible. That means we are trying to optimize this func.

- Optimization problem.

If we do,

Maximize variance $V = \frac{1}{n} \sum_{i=1}^{n} \left(\sum_{j=1}^{d} x_{ij} e_j \right)^2$

- There is a possibility of cheating here if some1 takes max. (huge) value of 'ej' and optimize the problem.
- To deal with this possibility of cheating, we can impose the constraint that the length of vector 'e' is unit length

$$[\,\|e\| = 1.\,] \quad \text{i.e.} \quad \left[\sum_{j=1}^{d} e_j^2 = 1 \right]$$

- Now, since the optimizat^n problem becomes a constraint optimizat^n, we need to add lagrange multiplier(α) to solve it.
- So, the problem becomes,

$$\left[\text{Maximize, } V = \frac{1}{n} \sum_{i=1}^{n} \left(\sum_{j=1}^{d} x_{ij} e_j \right)^2 - \alpha \left(\left(\sum_{j=1}^{d} e_j^2 \right) - 1 \right) \right]$$

taking derivative w.r.t. vector 'e' & 'a' is one of the att^r.

$$\frac{dV}{de_a} = \frac{2}{n} \sum_{i=1}^{n} \left(\sum_{j=1}^{d} x_{ij} e_j \right) . x_{ia} - 2\alpha e_a = 0.$$

$$\therefore \frac{2}{n} \sum_{j=1}^{d} e_j \underbrace{\left(\frac{1}{n} \sum_{i=1}^{n} x_{ia} \cdot x_{ij} \right)}_{\text{covariance of a,j}} = 2\alpha e_a \quad \text{for } a = 1, 2 \cdots d.$$

where, i ⇒ instances of data. & j ⇒ dimensions

$$= \left\{ \begin{array}{l} \sum_{j=1}^{d} \text{cov}(1,j)\, e_j = \alpha e_1 \\ \vdots \qquad\qquad \vdots \\ \sum_{j=1}^{d} \text{cov}(d,j)\, e_j = \alpha e_d. \end{array} \right\}$$

Vector for all values of $a = 1, 2 \cdots d$.

- If we observe this eqn, this is nothing but a 'dot' product of covariance matrix for all xov individually with vector 'ej'.
- This eqn can be represented as,

$$\boxed{\Sigma e = \lambda e}$$

Σ ⇑ covariance; e: eigenvector; λ: scalar multiplier or eigen value.

- This means that 'e' must be an eigen vector. Hence Proved that eigenvector is the vector which maximizes the variance.

★ Computing variance along the eigen vectors.

Variance of projected pts. ($x^T e$).

$$V = \frac{1}{n} \sum_{i=1}^{n} \left(\sum_{j=1}^{d} x_{ij} e_j - u \right)^2$$

$$u = \frac{1}{n} \cdot \sum_{i=1}^{n} \left(\sum_{j=1}^{d} x_{ij} e_j \right)$$

$$= \sum_{j=1}^{d} \cdot \left(\frac{1}{n} \sum_{i=1}^{n} x_{ij} \right) e_j$$

= mean of jth attr before the project^n. which is '0' because we centered the data initially.

$$\therefore \boxed{u = 0}$$

$$V = \frac{1}{n} \sum_{i=1}^{n} \left(\sum_{j=1}^{d} x_{ij} \cdot e_j \right)^2.$$

$$= \frac{1}{n} \sum_{i=1}^{n} \left(\sum_{j=1}^{d} x_{ij} \cdot e_j \right) \left(\sum_{a=1}^{d} x_{ia} \cdot e_a \right)$$

$$= \sum_{a=1}^{d} \sum_{j=1}^{d} \underbrace{\left(\frac{1}{n} \sum_{i=1}^{n} x_{ia} \, x_{ij} \right)}_{cov(a,j).} e_j \cdot e_a .$$

$$= \sum_{a=1}^{d} \underbrace{\left(\sum_{j=1}^{d} cov(a,j) \cdot e_j \right)}_{\lambda e_a .} e_a .$$

$$= \sum_{a=1}^{d} (\lambda e_a) \cdot e_a$$

$$= \lambda \sum_{a=1}^{d} e_a \cdot e_a .$$

$$= \lambda \, \|e\|^2$$

↑ This vector is unit length.

$$= \lambda \Leftarrow \text{Eigen Value.}$$

This proves that Eigenvalue is the value of variance along the eigen vectors.

- We got the proofs that eigenvectors are the direction in which data is spread & eigenvalue is the variance along this direction.

★ How many dimensions to pick?

- We start with higher no. of dimensions 'd' & then we pick smaller no. of dimensions 'm' where $m < d$. (i.e. we have eigenvectors. $e_1, e_2 \ldots e_d$ & we want to pick 'm' vectors from them where $m < d$.).
- we know that each eigen value is equal to the variance along e_i (i.e. $\lambda_i = e_i$). then the total variance = sum of all eigenvalues along all dimensions.

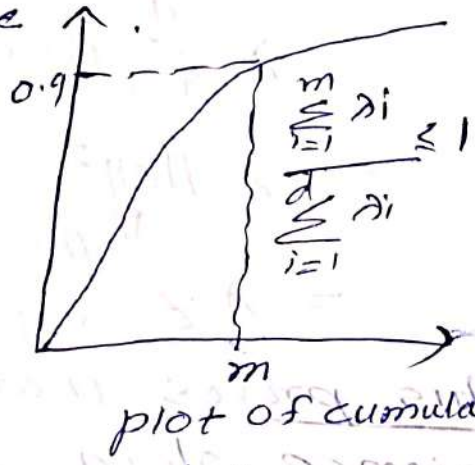
- On PCA, we try to preserve as much variance as possible. So, ~~our aim~~ to ~~get~~ achieve this, we need to select eigen vectors with maximum eigen values.

- Process for selecting the dimensions.
  - sort eigen~~value~~ vectors s.t $\lambda_1 \geq \lambda_2 \cdots \geq \lambda_d$.
  - Look at the cumulative sum plot
  - Pick up first 'm' eigenvectors which explain 90% or the total variance.
    - typical threshold values $\Rightarrow$ 0.9 or 0.95

$$\frac{\sum_{i=1}^{m} \lambda_i}{\sum_{i=1}^{d} \lambda_i} \leq 1$$

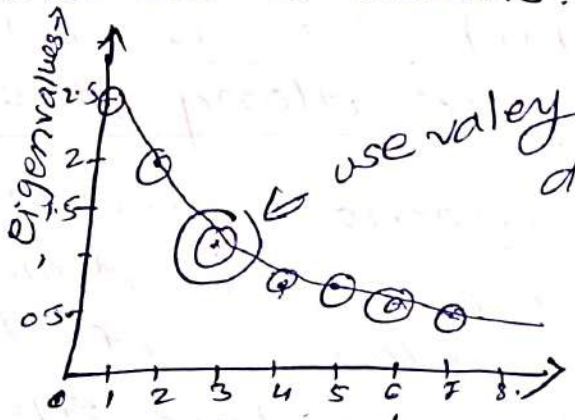
$\sum_{i=1}^{m} \lambda_i$ $\in$ ~~Cumulative sum~~.

$\sum_{i=1}^{d} \lambda_i$ $\in$ sum of total eigen values.



plot of cumulative sc...

- Method - II $\Rightarrow$ Use Scree plot
  - same as k-means.



use valey pt. as no. of dimensions.

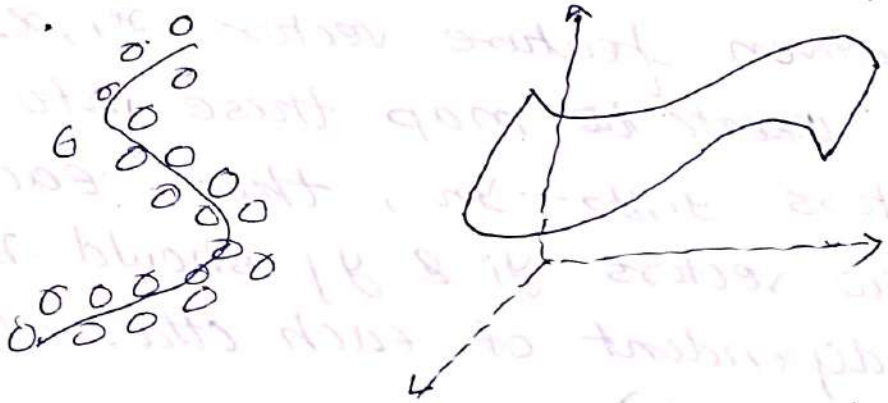
Practical Issues of PCA

1) Covariance is extremely sensitive to large values.
   - To deal with this, we can divide by standard deviation.

   $x' = (x - \text{mean}) / \text{standard deviation}$.

2) PCA assumes that the underlying subspace is linear. (i.e. line in 2D, flat sheet in 2D etc).
   - If our data is non linear then PCA doesn't work.

eg ⇒



Pros & Cons of dimensionality Reduction.

Pros -
- less space to save data. (reduction in the size of data).
- PCA rotates the space so that the dimensions become uncorrelated (independent).
  (eg ⇒ you can apply naive bayes on correlated data after applying PCA on it).

Cons
- Too expensive (can't be applied to very large datasets.
- Disastrous for tasks with fine grained classes.

- Assumes that the data is linearly seperated.

* Independent Component Analysis * refer Georgia tech Udacity 2 and NG.

- PCA deals with the idea of finding correlation by maximizing variance.
- ICA tries to maximize independence, i.e. it tries to find a linear xformation of our feature space into a new feature space s.t. each of the individual new features are mutually independent (in statistical space).
- Given feature vectors $x_1, x_2 \cdots x_n$, if we want to map these into new feature vectors $y_1, y_2 \cdots y_n$, then each one of the new vectors y_i & y_j should be stastically independent of each other. (i.e. their mutual info = 0).

$$[I(y_i, y_j) = 0]$$

- Also, PCA states that the mutual info. bet" all of the features ('y' & the original feature space (x) should be as high as possible.

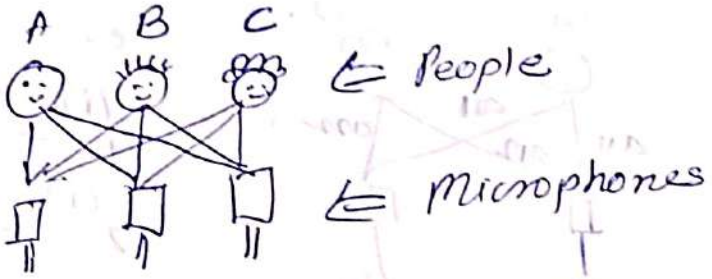
$$I(y; x) \Rightarrow \text{highest}$$

- Example.

- Given a set of mutually independent hidden variables & a set of observable states, the job is to find hiddenvariables given observables.

- Blind Source seperation Problem. / Cocktail Party Problem)

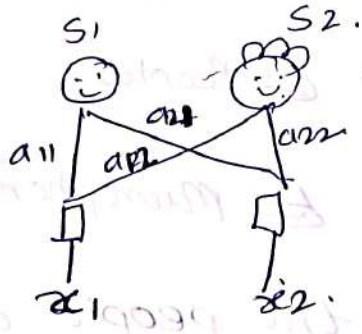
- In a party where there are lot of people talking independently, we as a listener can hear all mixed conversation.



- In this scenario, the people are hidden variables & microphones are observables.
- Here, each microphone is giving us a mixed signal. But, using ICA, we can extract the voices of A, B & C individually from these.
- Lets consider that in a canonical cocktail party, there are two speakers for simplicity. There are two microphones stationed in different locations in a party room.
- At the time instances $i = 1, 2 \cdots n$, the microphones provide us with voice measurements $x_1^{(i)}$ and $x_2^{(i)}$, such as amplitudes.
- In a party only two speakers are actually speaking & their speech signals are independent of each other.
- At time instant 'i', the speakers speak with the signal $s_1^{(i)}$ & $s_2^{(i)}$ respectively
- The microphones produce the combined voice from both the speakers.

(11) The goal of ICA is, given the time series data from microphones, to figure out original speakers' speech signals.

- The combination is assumed to be linear



i.e.

$$x_1^{(i)} = a_{11} s_1^{(i)} + a_{12} s_2^{(i)}$$
$$x_2^{(i)} = a_{21} s_1^{(i)} + a_{22} s_2^{(i)}$$

Where $a_{11}, a_{12}, a_{21}, a_{22}$ are unknown.

- If we consider 'n' microphones & 'n' speakers then the eqns can be represented as

$$\left[x^{(i)} = A s^{(i)} \right] \quad \text{i.e.} \quad \left[x = As \right] \quad x_j^{(i)} = \sum_k A_{jk} s_k^{(i)}$$

where A ← unknown, square, invertible mixing matrix.

- The goal is to recover unseen sources $s^{(i)}$

~~Bell & Sejnowski ICA Algorithm~~

i.e. to find w equal to A^{-1} so that

$$s^{(i)} = A^{-1} x^{(i)} \quad \text{i.e.} \left[s^{(i)} = w x^{(i)} \right]$$

$$w = \begin{bmatrix} w_1^T \\ w_2^T \\ \vdots \\ w_n^T \end{bmatrix}$$

- Lets assume that each of the speaker just emmits a white noise. for simplicity.
  i.e. $\left[s_j^{(i)} \sim \text{uniform } [-1, 1] \right]$
- The ambiguities in the signals (i.e. permutations & signs) are because the speaker signals are non guassians.

  $s_j^{(i)} \Rightarrow$ non guassians.
- ~~Given~~ Let speaker signals $s \in \mathbb{R}^n$.

  then density of s is $p_s(s)$..
- microphone recordings $x = As$.

  $= W^{-1}s$.

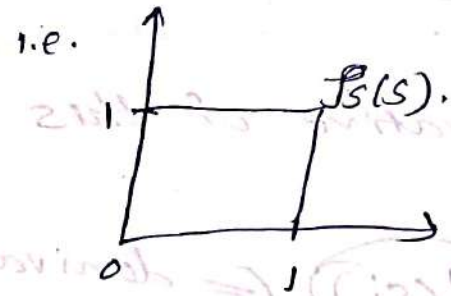
  $\therefore \left[s = Wx \right]$
- lets compute the density of x.

  $p_x(x) = p_s(Wx) \cdot |W|$.

eg- $p_s(s) = I\{0 \le s \le 1\}$ ($s \sim$ uniform $[0,1]$).

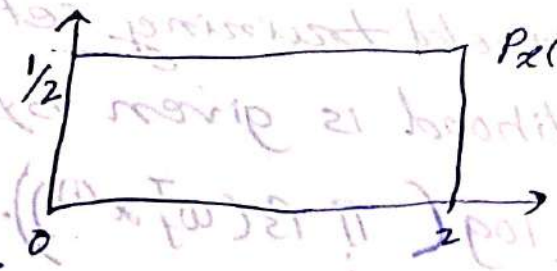
where $I \Rightarrow$ indicator func

i.e.



let $x = 2s$ where $A = 2$, $W = \frac{1}{2}$.

($x \sim$ uniform $[0,2]$).



$\therefore \left[p_x(x) = I\{0 \le x \le 2\} \cdot \frac{1}{2} \right]$

* ICA model.

$$p(s) = \prod_{i=1}^{n} p_s(s_i)$$. where s_i is independent for all n speakers

$$\therefore p(x) = \left[\prod_{i=1}^{n} p_s(w_i^T x) \right] \cdot |w| \cdot$$

$|w|$ ↑ determinant of w.

where $w = A^{-1} = \begin{bmatrix} -w_1^T- \\ -w_2^T- \\ \vdots \\ -w_n^T- \end{bmatrix}$ & $s_i = w_i^T x$.

- choose CDF for computing density $p_s(s_i)$.
- i.e. we need some func which increases from 0 to 1. Here, we cant choose guassian because the given signals are not guassian signals.
- therefore, lets select sigmoidal func for the same.

$$\therefore f(s) = \frac{1}{1+e^{-s}}.$$

- If we take derivative of this $f(s_i)$, it gives us $p_s(s_i)$.

$p_s(s_i) =$ $f'(s_i)$ ← derivative of $f(s)$.

- Algorithm -
  - Given, unlabeled training set, $\{x^{(1)} \dots x^{(n)}\}$, the log likelihood is given by.

$$l(w) = \sum_i \log \left(\prod_j p_s(w_j^T x^{(i)}) \cdot |w| \right)$$

  - using stochastic gradient ascent, where $p_s(s) = f'(s)$

$$w = w + \alpha \frac{d}{dw} l(w).$$

where $\frac{d}{dw} l(w) = \begin{bmatrix} 1 - 2g(w_1^T x) \\ \vdots \\ 1 - 2g(w_n^T x) \end{bmatrix} + (w^T)^{-1}.$

- Finally, $s^{(i)} = w \cdot x^{(i)}$.

* <u>Applications of ICA</u>
  - Processing of EEG data.